

Browser Console & Chrome DevTools — Tasks 1–15

Complete answers, JavaScript examples, expected output, and debugging steps

Note: Tasks that ask you to interact with the browser console or DevTools are answered with the exact code and steps to perform them. The PDF does not claim that those browser actions were actually executed on your device.

Task 1: Mock Instagram-style console notifications

Code:

```
console.log("Instagram: Post uploaded successfully!");
console.warn("Instagram: Caption is missing.");
console.error("Instagram: Image failed to upload.");
```

Expected result: Expected console: Instagram: Post uploaded successfully! ■ Instagram: Caption is missing. ✖ Instagram: Image failed to upload.

How to complete: Open Chrome DevTools with F12 or Ctrl+Shift+J, select Console, paste the code, and press Enter. The three messages appear with the usual log, warning, and error styling.

Task 2: Mock Instagram-style console notifications (same task as Task 1)

Code:

```
console.log("Instagram: Post uploaded successfully!");
console.warn("Instagram: Caption is missing.");
console.error("Instagram: Image failed to upload.");
```

Expected result: Expected output is the same as Task 1: a successful upload log, a caption warning, and an image-upload error.

How to complete: Run the same code in the browser Console. Task 2 duplicates Task 1.

Task 3: Display 5 songs with console.table()

Code:

```
const songs = [
  { title: "Blinding Lights", artist: "The Weeknd", duration: "3:20" },
  { title: "Shape of You", artist: "Ed Sheeran", duration: "3:53" },
  { title: "Levitating", artist: "Dua Lipa", duration: "3:23" },
  { title: "Perfect", artist: "Ed Sheeran", duration: "4:23" },
  { title: "Believer", artist: "Imagine Dragons", duration: "3:24" }
];

console.table(songs);
```

Expected result: Expected result: Chrome displays a formatted table with columns such as index, title, artist, and duration, with 5 rows.

How to complete: Save the code as songs.js and load it from an HTML page, or paste it directly into the browser Console.

Task 4: Display 5 food orders like a Zomato admin dashboard

Code:

```
const orders = [
```

```

    { orderId: "ZOM1001", itemName: "Paneer Pizza", price: 299 },
    { orderId: "ZOM1002", itemName: "Veg Biryani", price: 249 },
    { orderId: "ZOM1003", itemName: "Masala Dosa", price: 159 },
    { orderId: "ZOM1004", itemName: "Cheese Burger", price: 199 },
    { orderId: "ZOM1005", itemName: "Manchurian", price: 179 }
  ];

  console.table(orders);

```

Expected result: Expected result: A 5-row table showing orderId, itemName, and price.

How to complete: Paste the code into the Console. `console.table()` makes the order data easier to scan than normal `console.log()`.

Task 5: Display 5 food orders like a Zomato admin dashboard (duplicate task)

Code:

```

const orders = [
  { orderId: "ZOM1001", itemName: "Paneer Pizza", price: 299 },
  { orderId: "ZOM1002", itemName: "Veg Biryani", price: 249 },
  { orderId: "ZOM1003", itemName: "Masala Dosa", price: 159 },
  { orderId: "ZOM1004", itemName: "Cheese Burger", price: 199 },
  { orderId: "ZOM1005", itemName: "Manchurian", price: 179 }
];

console.table(orders);

```

Expected result: Expected result: The same 5-order formatted table as Task 4.

How to complete: Task 5 duplicates Task 4, so the same solution applies.

Task 6: addToCart() with log, warning, and error

Code:

```

function addToCart(productName, quantity) {
  if (quantity < 0) {
    console.error(`Error: Invalid quantity for ${productName}.`);
  } else if (quantity === 0) {
    console.warn(`Warning: Quantity for ${productName} is zero.`);
  } else {
    console.log(`Success: ${quantity} ${productName}(s) added to cart.`);
  }
}

addToCart("Pizza", 2);
addToCart("Burger", 0);
addToCart("Pasta", -1);

```

Expected result: Expected console: Success: 2 Pizza(s) added to cart. ■ Warning: Quantity for Burger is zero. ✕ Error: Invalid quantity for Pasta.

How to complete: The conditions are checked in order: negative quantity is an error, zero is a warning, and positive quantity is a success.

Task 7: Debug addToCart() with a breakpoint

Code:

```

function addToCart(item, quantity) {
  if (quantity === 0) {
    const message = item.toUpperCase(); // breakpoint here
  }
}

```

```

        throw new Error("Cannot add zero quantity");
    }
    console.log(`${quantity} ${item} added`);
}

addToCart("Pizza", 0);

```

Expected result: At the breakpoint, inspect: item = "Pizza" and quantity = 0. After continuing, the function throws: Error: Cannot add zero quantity.

How to complete: Open DevTools → Sources. Put a breakpoint on the line containing const message. Return to Console and run addToCart("Pizza", 0). Execution pauses before the error. In the Scope panel, inspect item and quantity. Use F8 to continue.

Task 8: Debug addToCart() with a breakpoint (duplicate task)

Code:

```

function addToCart(item, quantity) {
    if (quantity === 0) {
        const message = item.toUpperCase(); // breakpoint here
        throw new Error("Cannot add zero quantity");
    }
    console.log(`${quantity} ${item} added`);
}

addToCart("Pizza", 0);

```

Expected result: At the breakpoint: item = "Pizza" and quantity = 0. Continuing produces the intentional error.

How to complete: Task 8 duplicates Task 7. Use Sources, set the breakpoint, run addToCart("Pizza", 0), and inspect the local variables.

Task 9: Breakpoint inside a loop over friends

Code:

```

const friends = ["Aarav", "Diya", "Riya", "Kabir", "Meera"];

for (const friend of friends) {
    console.log(friend); // set breakpoint here
}

```

Expected result: Each time execution pauses, the loop variable friend contains the current name: Aarav, then Diya, then Riya, then Kabir, then Meera.

How to complete: Open Sources and set a breakpoint on console.log(friend). Run the script. Use F8 to continue to the next iteration and F10 to step over the current line. Inspect friend in the Scope panel.

Task 10: Identify the call that causes the Payment failed error

Code:

```

function placeOrder() {
    confirmOrder();
}

function confirmOrder() {
    processPayment();
}

function processPayment() {
    throw new Error("Payment failed");
}

```

```
}  
  
placeOrder();
```

Expected result: Root cause: processPayment() throws Error: Payment failed. The call stack is typically: processPayment → confirmOrder → placeOrder → global/script context.

How to complete: Open the Console and run the code. Expand the Error entry or click its source location. The stack trace shows that processPayment() is where the error is thrown, after it was called by confirmOrder(), which was called by placeOrder().

Task 11: Identify the call that causes the Payment failed error (duplicate task)

Code:

```
function placeOrder() {  
    confirmOrder();  
}  
  
function confirmOrder() {  
    processPayment();  
}  
  
function processPayment() {  
    throw new Error("Payment failed");  
}  
  
placeOrder();
```

Expected result: Root cause: processPayment() contains the throw statement. The call chain is placeOrder() → confirmOrder() → processPayment().

How to complete: Task 11 duplicates Task 10. Expand the console stack trace to see the same call chain.

Task 12: Find the function and line causing JSON.parse(undefined) to fail

Code:

```
function getUser() {  
    return fetchUser();  
}  
  
function fetchUser() {  
    return JSON.parse(undefined);  
}  
  
getUser();
```

Expected result: Function causing the error: fetchUser(). The failing statement is JSON.parse(undefined). The exact line number depends on where you paste/save the code, so it cannot be stated as a universal fixed number. In a 9-line script formatted exactly as shown, it is line 6.

How to complete: Run the code and expand the stack trace. It will point to fetchUser() at the JSON.parse(undefined) line, then show fetchUser() was called by getUser().

Task 13: Explain an error copied from the browser console

Code:

Example error:

```
Uncaught SyntaxError: Unexpected token '}'
```

Expected result: This means the JavaScript parser found a closing curly brace `}` where it was not syntactically expected. Common causes are an extra `}`, a missing opening `{`, or mismatched braces.

How to complete: To fix it, inspect the line named by the error, then check the surrounding if/else blocks, functions, loops, and objects. In DevTools, click the file/line link in the error to jump directly to the source.

Task 14: Explain a console error and suggest a fix (duplicate task)

Code:

```
Example error:  
Uncaught TypeError: Cannot read properties of undefined (reading 'name')
```

Expected result: This means code tried to access `.name` on a value that is undefined. For example, `user.name` fails if `user` is undefined.

How to complete: A fix is to ensure the object exists before accessing the property, for example: `if (user) console.log(user.name);`. The exact fix depends on why the value became undefined.

Task 15: Explain a real console error in your own words

Code:

```
Example real error from Task 10:  
Error: Payment failed
```

Expected result: In my own words: the program deliberately stops at `processPayment()` because that function throws an `Error`. The browser reports the error and shows the sequence of function calls that led to it.

How to complete: To fix a real application, inspect why payment processing failed and replace the intentional throw with appropriate validation, error handling, or recovery logic. For this practice task, the error is intentional so that the stack trace can be studied.

Quick DevTools Reference

Open Console: F12 or Ctrl+Shift+J (Chrome).

Sources: F12 → Sources. Find your JavaScript file, click the line number to set a breakpoint, then run the code.

F8: Continue/resume execution until the next breakpoint or completion.

F10: Step over the current line.

F11: Step into a function call.

Shift+F11: Step out of the current function.

Scope panel: Inspect local variables such as item, quantity, and loop variables while execution is paused.

Stack trace: Expand an Error in the Console to see which function threw the error and which calls led to it.

Submission note

For tasks asking for a real browser-console error, replace the example error in Tasks 13–15 with the exact error copied from your own DevTools Console, then explain it using the same structure: meaning → likely cause → fix.